Dynamic Memory Allocation in C

Last Updated : 23 Sep, 2025

Dynamic memory allocation techniques give programmer control of memory when to allocate, how much to allocate and when to de-allocate.

- We can allocate memory at runtime, allows us to handle data of varying sizes.
- The memory is allocated on the **heap** memory instead of the stack.
- The size of the memory can be increased if more elements are to be inserted and decreased of less elements are inserted.
- The allocated memory stays there (if the programmer has not de-allocated it) even after the function call is over. So a function can return pointer to the allocated memory. On the other hand, the memory/variables become invalid once the function call is over for normal variables allocated on stack.

Dynamic memory allocation is possible in C by using the following 4 library functions provided by **<stdlib.h>** library:

malloc()

The **malloc()** (stands for **m**emory **alloc**ation) function is used to allocate a single block of contiguous memory on the heap at runtime. The memory allocated by malloc() is uninitialized, meaning it contains garbage values.

Assume that we want to create an array to store 5 integers. Since the size of int is 4 bytes, we need 5 * 4 bytes = 20 bytes of memory. This can be done as shown:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(20);

    // Populate the array
    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;

    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
```

Output

```
1 2 3 4 5
```

In the above malloc call, we hardcoded the number of bytes we need to store 5 integers. But we know that the size of the integer in C depends on the architecture. So, it is better to use the **sizeof operator** to find the size of type you want to store.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(sizeof(int) * 5);

    // Populate the array
    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;

    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
```

```
}
```

Output

```
1 2 3 4 5
```

Moreover, if there is no memory available, the malloc will fail and return NULL. So, it is recommended to check for failure by comparing the ptr to NULL.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(sizeof(int) * 5);

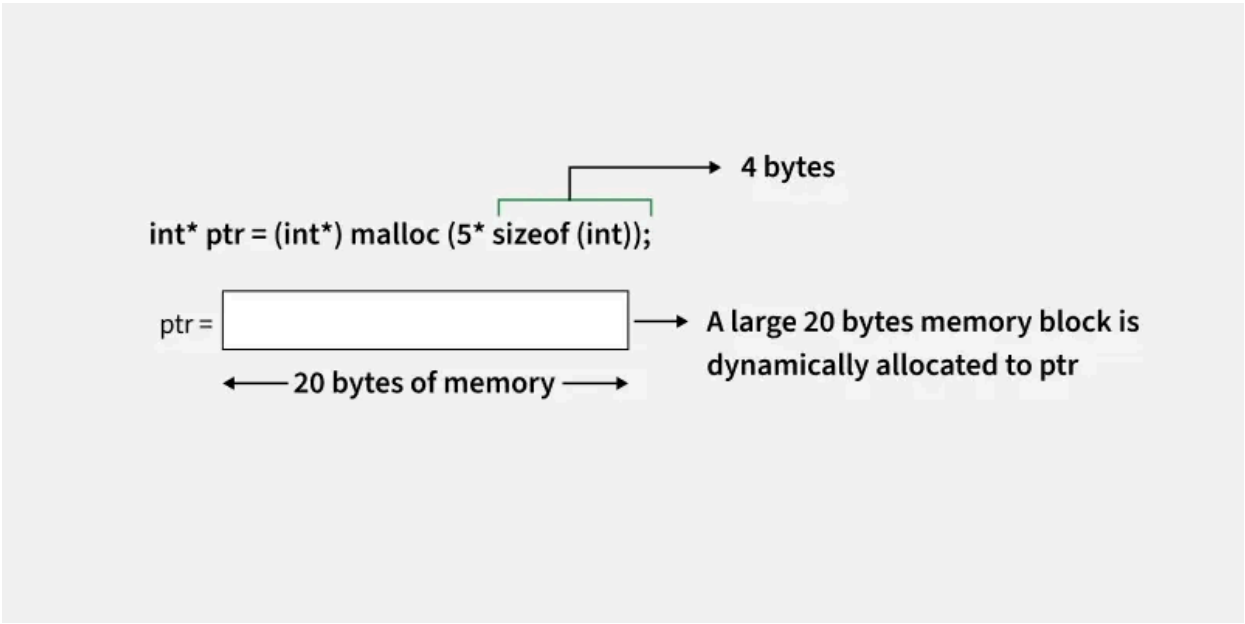
    // Checking if failed or pass
    if (ptr == NULL) {
        printf("Allocation Failed");
        exit(0);
    }

    // Populate the array
    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;

    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
```

Output

```
1 2 3 4 5
```



Syntax

```
malloc(size);
```

where **size** is the number of bytes to allocate.

This function returns a void pointer to the allocated memory that needs to be converted to the pointer of required type to be usable. If allocation fails, it returns NULL pointer.

calloc()

The **calloc()** (stands for **c**ontiguous **alloc**ation) function is similar to malloc(), but it initializes the allocated memory to zero. It is used when you need memory with default zero values.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)calloc(5, sizeof(int));

    // Checking if failed or pass
    if (ptr == NULL) {
        printf("Allocation Failed");
        exit(0);
```

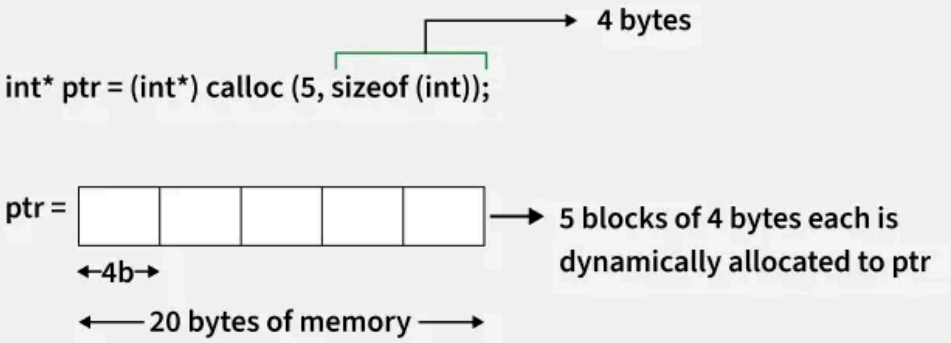
```
    }

    // No need to populate as already
    // initialized to 0

    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
```

Output

```
0 0 0 0 0
```



Syntax

```
calloc(n, size);
```

where **n** is the number of elements and **size** is the size of each element in bytes.

This function also returns a void pointer to the allocated memory that is converted to the pointer of required type to be usable. If allocation fails, it returns NULL pointer.

free()

The memory allocated using functions malloc() and calloc() is not de-allocated on their own. The **free()** function is used to release dynamically allocated memory back to the operating system. It is essential to free memory that is no longer needed to avoid memory leaks.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)calloc(5, sizeof(int));

    // Do some operations.....
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);

    // Free the memory after completing
    // operations
    free(ptr);

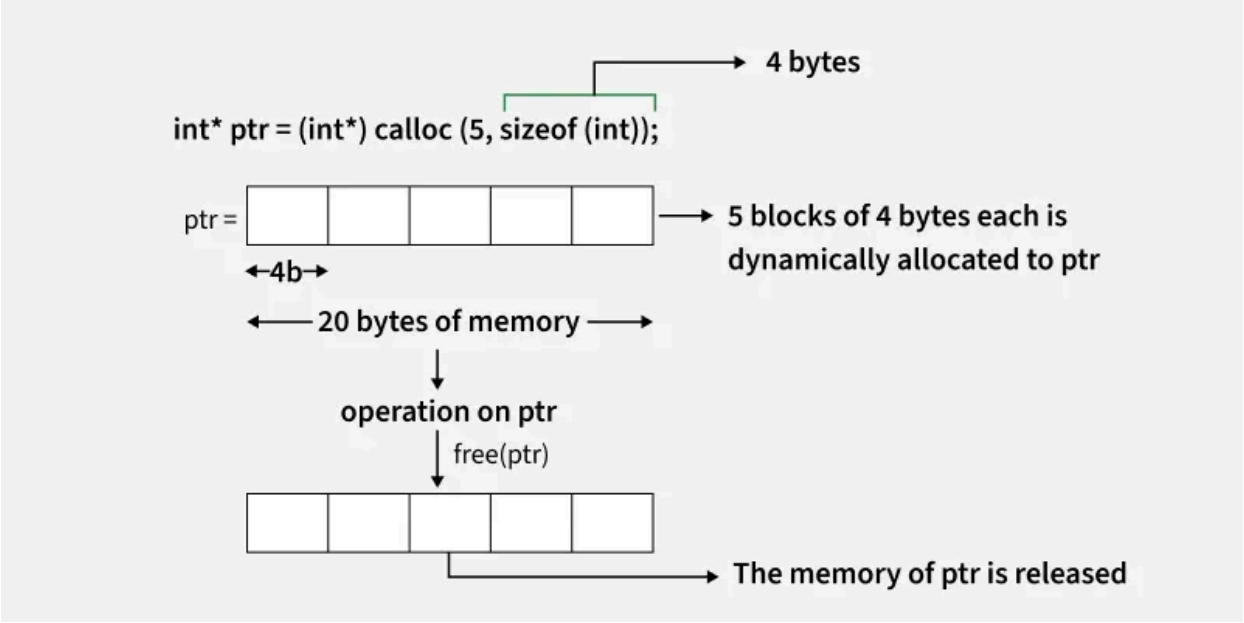
    return 0;
}
```

Output

```
0 0 0 0 0
```

After calling free(), it is a good practice to set the pointer to NULL to avoid using a "dangling pointer," which points to a memory location that has been deallocated.

```c
ptr = NULL;
```



Syntax

```
free(ptr);
```

where **ptr** is the pointer to the allocated memory.

After freeing a memory block, the pointer becomes invalid, and it is no longer pointing to a valid memory location.

realloc()

realloc() function is used to resize a previously allocated memory block. It allows you to change the size of an existing memory allocation without needing to free the old memory and allocate a new block.

Suppose we initially allocate memory for 5 integers but later need to expand the array to hold 10 integers. We can use realloc() to resize the memory block:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(5 * sizeof(int));

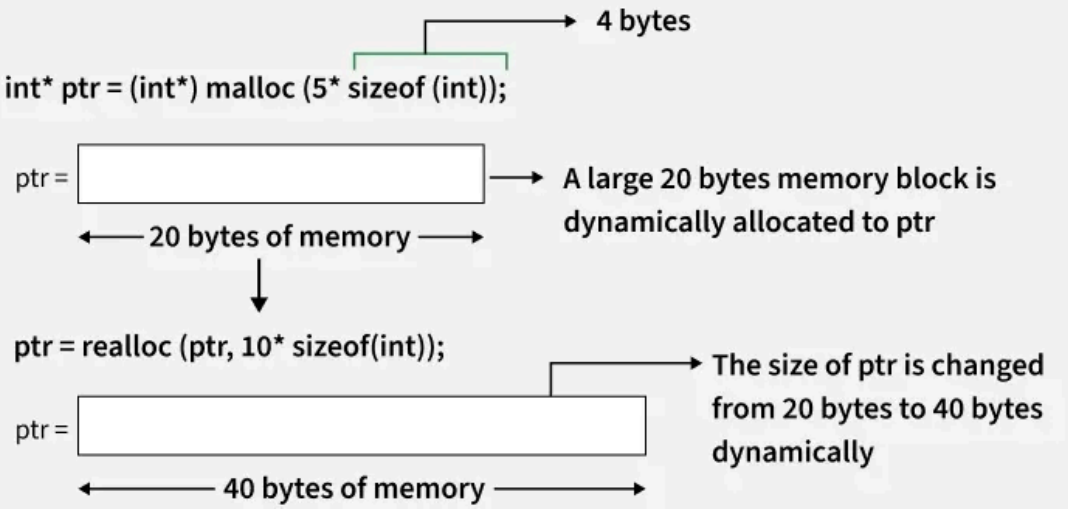
    // Resize the memory block to hold 10 integers
```

```
    ptr = (int *)realloc(ptr, 10 * sizeof(int));
```



```
    if (ptr == NULL) {
        printf("Memory Reallocation Failed");
        exit(0);
    }

    return 0;
}
```



It is important to note that if realloc() fails and returns NULL, the original memory block is not freed, so you should not overwrite the original pointer until you've successfully allocated a new block. To prevent memory leaks, it's a good practice to handle the NULL return value carefully:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(5 * sizeof(int));

    // Reallocation
    int *temp = (int *)realloc(ptr, 10 * sizeof(int));

    // Only update the pointer if reallocation is successful
```

```
    if (temp == NULL)
        printf("Memory Reallocation Failed\n");
    else
        ptr = temp;

    return 0;
}
```

Practical Example

Consider the first scenario where we were having issues with the fixes size array. Let's see how we can resolve both of these issues using dynamic memory allocation.

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    // Initially allocate memory for 5 integers
    int *ptr = (int *)malloc(5 * sizeof(int));

    // Check if allocation was successful
    if (ptr == NULL) {
        printf("Memory Allocation Failed\n");
        exit(0);
    }

    // Now, we need to store 8 elements so
    // Reallocate to store 8 integers
    ptr = (int *)realloc(ptr, 8 * sizeof(int));

    // Check if reallocation was successful
    if (ptr == NULL) {
        printf("Memory Reallocation Failed\n");
        exit(0);
    }

    // Assume we only use 5 elements now
    for (int i = 0; i < 5; i++) {
        ptr[i] = (i + 1) * 10;
```

```
    }

    // Shrink the array back to 5 elements
    ptr = (int *)realloc(ptr, 5 * sizeof(int));

    // Check if shrinking was successful
    if (ptr == NULL) {
        printf("Memory Reallocation Failed\n");
        exit(0);
    }

    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);

    // Finally, free the memory when done
    free(ptr);

    return 0;
}
```

Output

```
10 20 30 40 50
```

In this program, we are managing the memory allocated to the pointer **ptr** according to our needs by changing the size using realloc(). It can be a fun exercise to implement an array which grows according to the elements inserted in it. This kind of arrays are called **dynamically growing arrays**.

Issues Associated with Dynamic Memory Allocation

As useful as dynamic memory allocation is, it is also prone to errors that requires careful handling to avoid the high memory usage or even system crashes. Few of the common errors are given below:

- **Memory Leaks**: Failing to free dynamically allocated memory leads to memory leaks, exhausting system resources.

- **Dangling Pointers**: Using a pointer after freeing its memory can cause undefined behavior or crashes.
- **Fragmentation**: Repeated allocations and deallocations can fragment memory, causing inefficient use of heap space.
- **Allocation Failures**: If memory allocation fails, the program may crash unless the error is handled properly.

malloc() vs calloc()

The functions malloc() and calloc() works very similar to one another. So, why there was the need for two such similar functions.

It turns out that even though they are similar, they have different use cases due to the minor difference between them regarding the memory initialization. malloc() does not initialize memory while calloc() initializes the memory with zero.

Dynamic Memory Allocation (malloc(), calloc(), and free())

Visit Course

Comment | More info

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play | Download on the App Store

Company

About Us
Legal
Privacy Policy
Careers
Contact Us
Corporate Solution
Campus Training Program

Explore

POTD
Job-A-Thon
Connect
Blogs
Nation Skill Up

Tutorials

Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
GATE
School Subjects
Software and Tools

Courses

IBM Certification
DSA and Placements
Web Development
Data Science
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Offline Centers

Noida
Bengaluru
Pune

Preparation Corner

Interview Corner
Aptitude
Puzzles

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved